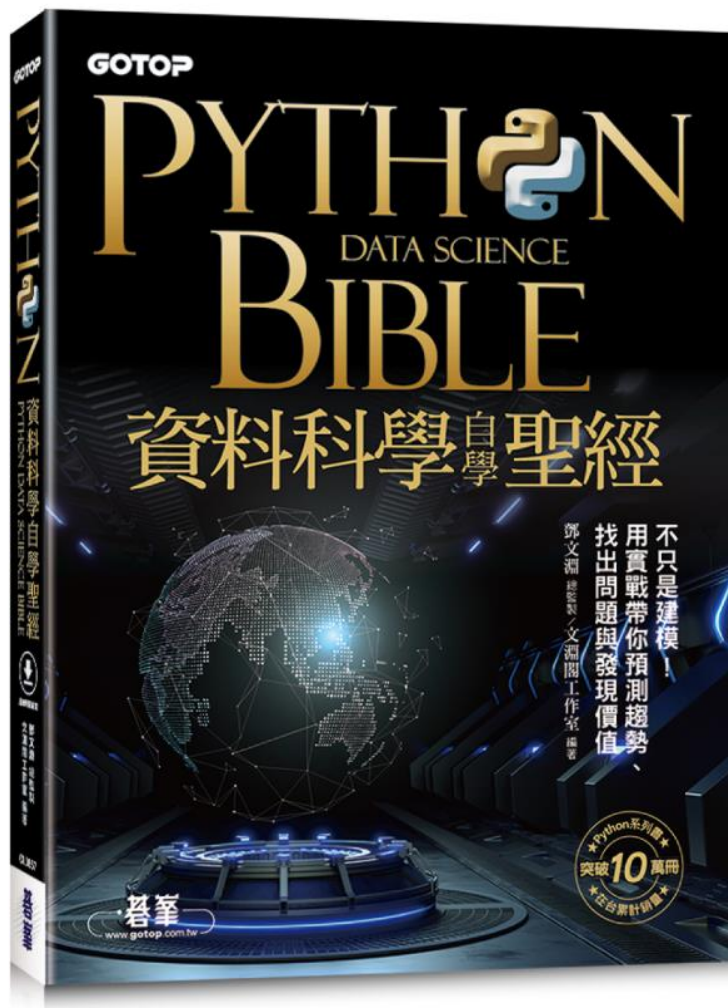




# PYTHON DATA SCIENCE BIBLE 資料科學自學聖經

**碁峯資訊**

【投影片使用規範與聲明】本投影片僅供非營利教學用途，教師得搭配用書授課講解使用，可於用書期間內將投影片置放於學校內部網站，但需有帳密權限機制，且僅供修課學生瀏覽使用。敬請老師善盡著作權保護之責，請勿將投影片任意散布與販售，亦不得以任何形式或方法轉載內容使用。

## 資料預處理： 標準化、資料轉換與特徵選擇

[6.1 Scikit-Learn：機器學習的開發工具](#) [6.4 認識特徵選擇](#)

[6.2 數值資料標準化](#)

[6.5 使用Pandas 進行特徵選擇](#)

[6.3 非數值資料轉換](#)

[6.6 使用Scikit-Learn 進行特徵選擇](#)

## 6.1 Scikit-Learn：機器學習的開發工具

### 6.1.1 認識Scikit-Learn

Scikit-Learn 建構在NumPy、SciPy 與Matplotlib 之上，是開源並可作為商業使用的模組。

Scikit-Learn 主要的撰寫程式語言是Python，在其中廣泛使用NumPy 進行線性代數及陣列運算，而且部分核心演算法以Cython 撰寫，因此運算效能極高。

Scikit-Learn基本上是用CPU 訓練模型，所以Scikit-Learn 相當適合入門的機器學習，不必進行繁瑣的GPU 環境設定。

### 6.1.2 Scikit-Learn 主要功能

Scikit-Learn 主要功能分為六類：

#### 分類(Classification)

- 功能：確認檢測的目標屬於哪一種類別。
- 應用：垃圾郵件偵測、圖像識別等。
- 演算法：支援向量機\_分類(SVC)、K近鄰(K-Neighbors) 等。

## 迴歸(Regression)

- 功能：預測檢測資料的連續值屬性。
- 應用：藥物反應(Drug response)、股價預測 (Stock prices) 等。
- 演算法：支援向量機\_迴歸(SVR) 等。

## 分群(Clustering)

- 功能：自動將資料分類成不同群集。
- 應用：客戶分類(Customer segmentation) 等。
- 演算法：K-Means 等。

## 降維(Dimensionality reduction)

- 功能：減少要考慮的特徵數量。
- 應用：視覺化(Visualization)、提高效率 (Increased efficiency) 等。
- 演算法：主成份分析(PCA) 等。

## 模型選擇(Model selection)

- 功能：比較、驗證各參數與模型。
- 應用：通過調整參數提高準確度。
- 演算法：網格搜索(Grid search)、交叉驗證 (Cross validation) 等。

## 預處理(Preprocessing)

- 功能：特徵提取與標準化。
- 應用：前處理(Preprocessing)、特徵擷取 (Feature extraction)。
- 演算法：Z分數標準化(StandardScaler)、最大最小值標準化(MinMaxScaler) 等。

## 6.1.3 Scikit-Learn 核心理念

- **一致性**：一致性是指 Scikit-Learn 定義的類別都具有相同的 API 介面，例如進行資料預處理的轉換器都具備「fit\_transform」方法；進行資料預測的預測器都具備「predict」方法。
- **可檢驗性**：可檢驗性是指 Scikit-Learn 定義的類別所依據的參數及結果，都可以透過屬性擷取出來檢視。
- **標準類**：標準類是指輸入與輸出的資料型態或結構，都是以內建資料與 Numpy 陣列來處理，不必自行創建類別。
- **模組化**：模組化是指可將 Scikit-Learn 的類別進行組裝，例如將轉換器與預測器組裝成為一個稱為管線(Pipeline)的類別。
- **預設值**：預設值是指在建立功能模組時，都會使用一組預設參數作為初始化的依據，而這些依據通常是多數使用者習慣的參數設計。

## 6.2 數值資料標準化

在機器學習與深度學習中，資料集的欄位資料稱為「特徵」。當特徵的單位或者大小相差較大，就容易影響訓練結果，使得一些演算法無法學習到其他特徵，所以要將不同規格的資料轉換到同一規格。透過一些轉換運算將特徵資料轉換成更加適合演算法的特徵資料的過程，稱為「數值資料標準化」，常用的方法有Z 分數標準化、最大最小值標準化、最大絕對值標準化 及RobustScaler 標準化。

數值資料標準化後，各特徵值的數值大小都差不多，所以每一個特徵值的重要性就幾乎相同了！

### 6.2.1 Z 分數標準化：StandardScaler

Z 分數標準化是使用最多的數值資料標準化方法。

#### Z 分數標準化相關公式

Z 分數標準化的轉換公式為：

$$Y = \frac{X - X_{avg}}{S}$$

Y 是轉換後的數值，X 是原始數值，Xavg 是所有數值的平均數，S 是標準差。標準差(S) 的計算公式為：

$$S = \sqrt{\frac{\sum (X - X_{avg})^2}{n}}$$

「Σ」是總和。

而Z 分數標準化轉換後的數值的意義，就是「原始數值」與「平均值」的差距是多少個「標準差」。

## Scikit-Learn 的Z 分數標準化模組

安裝Scikit-Learn 模組的語法為：

```
!pip install sklearn
```

要使用Z 分數標準化轉換，首先載入Scikit-Learn 的Z 分數標準化模組：

```
from sklearn.preprocessing import StandardScaler
```

接著建立StandardScaler 物件，語法為：

```
Z 分數物件 = StandardScaler()
```



然後用StandardScaler 物件的fit\_transform() 方法轉換，語法為：

轉換物件 = Z 分數物件.fit\_transform( 數值資料 )

## 6.2.2 最大最小值標準化：MinMaxScaler

最大最小值標準化是透過對原始數值資料進行變換，把資料轉換為0 到1 之間的數值資料。

### 最大最小值標準化相關公式

最大最小值標準化的轉換公式為：

$$Y = \frac{(X - X_{min})}{(X_{max} - X_{min})}$$

Y 是轉換後的數值，X 是原始數值， $X_{min}$  是資料最小數值， $X_{max}$  是資料最大數值。因為「 $X - X_{min}$ 」必定小於或等於「 $X_{max} - X_{min}$ 」，所以轉換值在0 到1 之間。

轉換後的數值是最大值轉換為1，最小值轉換為0，其餘數值在1 與0 之間。

### Scikit-Learn 的最大最小值標準化模組

使用Scikit-Learn 模組進行最大最小值標準化轉換，使用方法與Z 分數標準化完全相同，只要將StandardScaler 改為MinMaxScaler 即可。

## 6.2.3 最大絕對值標準化：MaxAbsScaler

最大絕對值標準化是耗費計算資源最少的數值資料標準化方法，即轉換的速度最快。

### 最大絕對值標準化相關公式

最大絕對值標準化與最大最小值標準化原理接近，不同處是最大絕對值標準化會保留原始數值資料正負符號，把資料轉換為-1 到1 之間的數值。

最大絕對值標準化的轉換公式為：

$$Y = \frac{X}{|X|_{max}}$$

Y 是轉換後的數值，X 是原始數值， $|X|_{max}$  是資料絕對值的最大數值。因為原始資料保留正負號，且X 小於等於 $|X|_{max}$ ，所以轉換值在-1 到1 之間。

### Scikit-Learn 的最大絕對值標準化模組

使用Scikit-Learn 模組進行最大絕對值標準化轉換，使用方法與Z 分數標準化完全相同，只要將StandardScaler 改為MaxAbsScaler 即可。

## 6.2.4 RobustScaler 標準化：RobustScaler

RobustScaler 標準化是根據資料的中位數及四分位數來轉換數值資料，此方法適用於包含異常值的資料。

### 最大絕對值標準化相關公式

RobustScaler 標準化的轉換公式為：

$$Y = \frac{(X - X_{median})}{IQR}$$

Y 是轉換後的數值，X 是原始數值，Xmedian 是資料的中位數，IQR 為資料的四分位間距。

### Scikit-Learn 的RobustScaler 標準化模組

使用Scikit-Learn 模組進行RobustScaler 標準化轉換，使用方法與Z 分數標準化完全相同，只要將StandardScaler 改為RobustScaler 即可。

## 6.2.5 數值資料標準化方法的比較

### Z 分數標準化

- **優點**：受異常資料的影響較小，因為標準化的運算基礎是平均值及標準差，少數異常資料經過平均運算後，平均值及標準差的數值不會有太大改變。
- **缺點**：大多數使用者對於標準差的意義較難理解，因此對於 Z 分數標準化的原理不易了解。還有計算過程較為繁雜，因此運算耗費的時間較長且較耗費資源，尤其是資料數量龐大時要考慮電腦計算資源。
- **適用範圍**：常態分布資料。因一般資料多為常態分布，適用範圍最廣，是使用最多的標準化方法。

### 最大最小值標準化

- **優點**：原理簡單，易於了解，運算速率較快。
- **缺點**：容易受異常資料影響，若有幾個數值很大或很小的異常值，就會造成最大值或最小值大幅失真，導致整個轉換資料都產生很大誤差。
- **適用範圍**：沒有異常資料的資料。在下列兩種情境多會使用最大最小值標準化：資料及特徵數量不多時，可用人工判斷有無異常資料；或者是處理圖形資料時，因圖形特徵為像素，而像素特徵值通常在0 到255 之間，不會有異常資料。

## 最大絕對值標準化

- **優點**：原理最簡單，運算速率最快。
- **缺點**：容易受絕對值很大數值的異常資料影響，因為若有幾個絕對值數值很大的異常值，會造成最大絕對值失真，導致整個轉換資料產生很大的誤差。
- **適用範圍**：沒有異常資料且需要保留正負值的資料。例如包含損益金額的資料集，需由正負值判斷是虧損還是獲利。

## RobustScaler 標準化

- **優點**：受異常資料的影響最小，因為四分位數及四分位間距轉換可縮小異常值與正常值的差距。
- **缺點**：原理最不易了解。運算耗費的時間較最大最小值標準化長，但比 Z 分數標準化短。
- **適用範圍**：包含較多異常值資料。

## 6.3 非數值資料轉換

機器學習演算法是一種數學模型，需在「數值」數據基礎上進行運算。

非數值資料需經預先處理轉換為數值資料後，才能傳送給機器學習演算法進行處理。

將非數值資料轉換為數值資料常用的方法有 **對應字典法**、**標籤編碼法** 及 **One-Hot 編碼法** 三種。

### 6.3.1 對應字典法

如果要轉換的特徵值不多，可用 **對應字典法** 直接將文字替換為數值。對應字典法是先將要替換的文字與對應的數值建立成字典，再以Dataframe 的df.replace() 或df.map() 方法進行轉換。

#### 資料取代：df.replace()

建立特徵值與對應數值的字典，語法為：

```
字典變數 = {'特徵值 1': 數值 1, '特徵值 2': 數值 2, ……}
```

最後即可使用`df.replace()` 方法轉換為數值資料，語法為：

```
DataFrame 欄位.replace(字典變數, inplace=True)
```

在機器學習實作時，進行預測所得的類別結果是數值，使用者常需要將數值資料還原為對應的文字。要達成此目的，只要將字典改為數值對應文字型式即可還原：

```
字典變數 = {數值 1:'特徵值 1', 數值 2:'特徵值 2', .....}
```

## 資料對應：`df.map()`

語法為：

```
DataFrame 欄位 = DataFrame 欄位.map(字典變數)
```

## 6.3.2 標籤編碼法

對應字典法原理簡單易懂，但如果特徵值數量很多，例如「工作」特徵多達12 個特徵值，建立字典會耗費很多時間。

**標籤編碼法** 會自動偵測所有特徵值，將N 個特徵值以0 到N-1 數值取代。

## Scikit-Learn 的LabelEncoder 模組

Scikit-Learn 提供LabelEncoder 模組進行標籤編碼法轉換。

首先要載入模組，語法為：

```
from sklearn.preprocessing import LabelEncoder
```

然後建立LabelEncoder 物件，語法為：

```
標籤物件 = LabelEncoder()
```

最後以標籤物件的fit\_transform() 進行轉換：

```
DataFrame 欄位 = 標籤物件.fit_transform(DataFrame 欄位)
```

## 查詢轉換後數值代表特徵值

標籤編碼法自動將特徵值轉換為數值，如果想查詢數值所代表的特徵值，可以使用LabelEncoder 物件的classes\_ 屬性，語法為：

```
標籤物件.classes_
```



## 將轉換後的數值還原為特徵值

LabelEncoder 物件的inverse\_transform() 方法可將數值還原為文字特徵值，

語法為：`DataFrame 欄位 = 標籤物件變數.inverse_transform(DataFrame 欄位)`

## 批次轉換所有非數值特徵

標籤編碼法可對指定特徵自動進行數值轉換，若能取得所有非數值特徵，利用迴圈就可一次對所有非數值特徵進行數值轉換。

取得所有非數值特徵的語法為：`欄位變數 = DataFrame 變數.select_dtypes(exclude=[np.number]).columns`

接著利用迴圈逐一對非數值特徵進行數值轉換，並顯示特徵值：

```
1 for i in cols:
2     df[i] = label1.fit_transform(df[i])
3     print('「{}」特徵對照：'.format(i), end='')
4     for j in range(len(label1.classes_)):
5         print('{} - {}'.format(j, label1.classes_[j]), end=', ')
6     print()
7 df
```

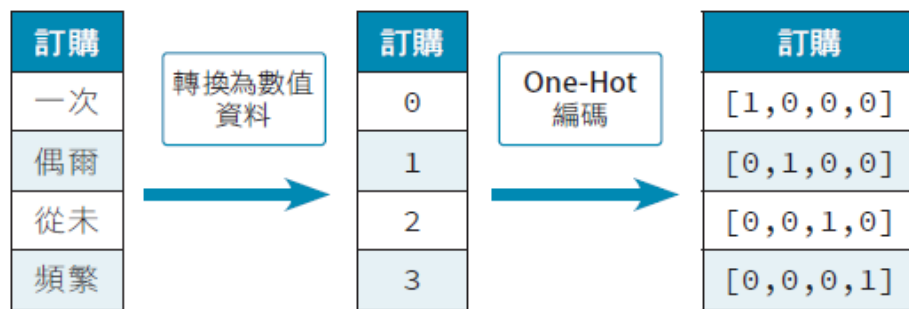
「工作」特徵對照：0 - 0, 1 - 1, 2 - 2, 3 - 3, 4 - 4, 5 - 5, 6 - 6, 7 - 7, 8 - 8, 9 - 9, 10 - 10, 11 - 11,  
「婚姻」特徵對照：0 - 0, 1 - 1, 2 - 2, 3 - 3,  
「學歷」特徵對照：0 - 0, 1 - 1, 2 - 2, 3 - 3, 4 - 4, 5 - 5, 6 - 6,  
「聯絡方式」特徵對照：0 - 0, 1 - 1,  
「訂購」特徵對照：0 - 0, 1 - 1, 2 - 2, 3 - 3,

|   | 年齡   | 工作 | 婚姻 | 學歷 | 聯絡方式 | 最後聯絡時間 | 聯絡次數 | 消費價格指數 | 消費信心指數 | 訂購 |
|---|------|----|----|----|------|--------|------|--------|--------|----|
| 0 | 31.0 | 5  | 0  | 6  | 1    | 750.0  | 2    | 92.893 | -46.2  | 0  |
| 1 | 55.0 | 7  | 1  | 6  | 0    | 154.0  | 8    | 94.465 | -41.8  | 0  |
| 2 | 27.0 | 0  | 1  | 2  | 1    | 466.0  | 1    | 92.893 | -46.2  | 3  |
| 3 | 35.0 | 0  | 1  | 1  | 1    | 222.0  | 1    | 93.075 | -47.1  | 0  |

### 6.3.3 One-Hot 編碼法

在機器學習中，One-Hot 編碼在指定的串列或陣列裡，只能有一個元素是1，其他元素都必須是0。例如，範例中的「訂購」特徵分類成4個值：一次、偶爾、從未、頻繁。

經過數值資料轉換後，就變成0、1、2、3。但若要轉成One-Hot 編碼，每個值就是包含4個元素的串列，0為第1個元素為1，其餘為0；1為第2個元素為1，其餘為0，依此類推：



機器學習中的類別資料多採用One-Hot 編碼，其最大優點是可消除數值的大小關係，讓每一個元素都處於相等地位。

Scikit-Learn 提供OneHotEncoder 模組進行One-Hot 編碼法轉換。

1. 首先要載入模組，語法為：

```
from sklearn.preprocessing import OneHotEncoder
```

2. 然後建立OneHotEncoder 物件，語法為：

```
onehot 物件 = OneHotEncoder(sparse= 布林值 )
```

■ sparse：參數值若為 True 表示建立 sparse 格式，若為 False 表示建立串列格式，預設值為True。sparse 格式較節省記憶體，但可讀性較差，不易理解。

3. 最後以fit\_transform() 方法進行One-Hot 轉換：

```
陣列變數 = onehot 物件 .fit_transform(DataFrame 欄位 )
```

若要在Pandas 中觀察One-Hot 編碼，則需使用OneHotEncoder 物件的get\_feature\_names\_out() 方法轉為DataFrame，語法為：

```
DataFrame 變數 = pd.DataFrame( 陣列變數 ,  
                                columns=onehot.get_feature_names_out( 特徵名 ))
```

## 6.4 認識特徵選擇

### 什麼是特徵選擇？

「特徵選擇」就是盡量在無損於機器學習演算法效能的情況下，過濾掉沒有效用、不具有關鍵影響力，以及有著重複或類似鑑別能力的雜訊特徵，最後僅保留下真正對效能指標有影響的特徵。

特徵選擇具有下列優點：

- 降低資料量，提升機器學習效能。
- 簡化模型，使模型更容易理解。
- 增加模型預測準確率。
- 改善模型通用性，降低過擬合風險。

## 如何選擇重要的特徵？

相關係數 可以指出兩組資料之間的相關性，對於選擇使用哪些特徵來進行機器學習很有幫助。相關係數的絕對值越大表示相關性越大，0 則表示兩個特徵沒有相關。

正相關是當一個特徵值增加時，另一個特徵值也增加；負相關是當一個特徵值增加時，另一個特徵值會減小。

通常相關強度與相關係數絕對值的關係：

| 相關係數絕對值 | 相關強度     |
|---------|----------|
| 0.8~1.0 | 極強相關     |
| 0.6~0.8 | 強相關      |
| 0.4~0.6 | 中等相關     |
| 0.2~0.4 | 弱相關      |
| 0.0~0.2 | 極弱相關或無相關 |

## 6.5 使用Pandas 進行特徵選擇

Pandas 提供三種計算相關係數的方法：皮爾森(Pearson)、肯德爾(Kendall) 及斯皮爾曼(Spearman)，這三種方法都適用於迴歸分析(即目標為數值的分析)，例如房價、股價預測。

### 6.5.1 使用Pandas 計算相關係數

Pandas 計算相關係數的語法為：

```
相關變數 = DataFrame 變數 .corr(method= 計算方式 , min_periods= 數值 )
```

- **method**：設定使用的計算相關係數方法：pearson(皮爾森相關係數)為預設值、kendall(肯德爾相關係數)、spearman(斯皮爾曼相關係數)。
- **min\_periods**：設定最小資料數量。預設值為 1。

## 6.5.2 皮爾森(Pearson) 相關係數

皮爾森相關係數 的意義為兩個特徵的共變異數與標準差乘積的關係。

皮爾森相關係數的值在-1 與1 之間，其絕對值越大表示相關性越大，0 則表示兩個特徵沒有相關。皮爾森相關係數值越接近1 表示正相關越強，越接近-1 表示負相關越強。皮爾森相關係數適用於線性分布且常態分布的特徵值。

Pandas 計算皮爾森相關係數的語法為：

```
相關變數 = DataFrame 變數.corr(method='pearson', min_periods= 數值)
```

使用者可以觀察相關係數手動進行特徵選擇(即選取相關係數絕對值較大者)，若特徵數量極多，進行特徵選擇會耗費不少時間，並且可能因疏忽導致選取錯誤。

以下程式可選取中等以上相關的特徵：

```
[3] 1 targetCorr = featuresCorr['房價']
    2 targetCorr = targetCorr.drop('房價')
    3 selectedFeatures = targetCorr[abs(targetCorr) > 0.4]
    4 print("選擇特徵數： {} \n選擇特徵:\n{}".
    5       format(len(selectedFeatures), selectedFeatures))
```

```
選擇特徵數： 6
選擇特徵：
公設比      -0.478614
NO濃度      -0.422592
房間數       0.696515
繳稅率      -0.464384
師生比      -0.504142
低收入比    -0.735179
Name: 房價, dtype: float64
```

### 6.5.3 肯德爾(Kendall) 相關係數

**肯德爾相關係數** 是按特定特徵排序，其他特徵通常是亂序的，此時計算同序對和異序對的差異來計算肯德爾相關係數。肯德爾相關係數的值也在-1 與1 之間，其意義與皮爾森相關係數相同。**肯德爾相關係數適用於非線性分布並且資料數量較小的特徵值。**

Pandas 計算肯德爾相關係數的語法為：

```
相關變數 = DataFrame 變數 .corr(method='kendall', min_periods= 數值 )
```

### 6.5.4 斯皮爾曼(Spearman) 相關係數

**斯皮爾曼相關係數** 是將兩個特徵值分別依大小排序後成對等級，再以各對等級差來進行計算而得到。斯皮爾曼相關係數的計算速度比肯德爾相關係數值快。斯皮爾曼相關係數的值也在-1 與1 之間，其意義與皮爾森相關係數相同。**斯皮爾曼相關係數適用於非線性分布並且具有異常值的特徵值。**

Pandas 計算斯皮爾曼相關係數的語法為：

```
相關變數 = DataFrame 變數 .corr(method='spearman', min_periods= 數值 )
```



## 6.6 使用Scikit-Learn 進行特徵選擇

Scikit-Learn 也提供多種篩選特徵的方法，在這裡以最常使用的卡方驗證進行說明。卡方驗證法適用於分類問題的分析，例如申請房貸是否核准、是否罹患癌症等。

### 認識卡方驗證

卡方驗證是用來處理分類並以計次資料的統計方法。計算方式是以觀察值及期望值的差異來得到卡方值，進而判斷兩特徵的相關程度：通常卡方值越大則兩特徵的相關程度越高。

### 載入Scikit-Learn 的卡方驗證模組

在Scikit-Learn 中使用卡方驗證要先載入模組：

```
from sklearn.feature_selection import SelectKBest  
from sklearn.feature_selection import chi2
```

## 取出判斷特徵與目標特徵

Scikit-Learn 的卡方驗證使用判斷特徵及目標特徵做為參數。在這個範例中，希望使用前9 個特徵來判別第10 個特徵，也就是「種類」欄的資料是良性或惡性。

1. **建立判斷特徵**: 取出資料集前9 個特徵當作判斷特徵。這裡使用`df.iloc()` 方法取得特徵資料，語法為：

```
DataFrame 變數 .iloc[ 列範圍 , 行範圍 ]
```

## 進行卡方驗證

使用Scikit-Learn 模組卡方驗證功能，首先是建立SelectKBest 物件，語法為：

```
卡方物件 = SelectKBest(chi2, k= 數值 )
```

- `k`：設定篩選後的特徵數量，預設值為10。

然後利用卡方變數的`fit_transform` 方法進行特徵篩選，語法為：

```
卡方陣列變數 = 卡方物件 .fit_transform( 判斷特徵 , 目標特徵 )
```

## 取得特徵驗證值

若是要以程式取得篩選出的特徵名稱，第一步是以卡方變數的scores\_ 屬性得到所有特徵的卡方驗證值，語法為：

```
卡方值變數 = 卡方變數 .scores_
```

## 取得相關特徵名稱

卡方驗證值越大則相關性越高，因此這裡要取出卡方驗證值前5 大的特徵名稱。這裡可以使用numpy 的argsort() 函式，可將數值由小到大排序，再使用flipud() 函式將陣列反轉變成由大到小排序(argsort() 函式無法由大到小排序)，再取出原始陣列的索引值組成陣列，語法為：

```
排序變數 = np.argsort(卡方值變數)  
排序變數 = np.flipud(排序變數)
```